AI Development Roadmap
Phase One: Basic Navigation
Objective: Implement basic navigation for AI agents.

Create a simple AI using Unity's navigation system.
Move the agent to a specified goal point.
Navigate around obstacles.
Success Criteria: The agent can reach the goal point while avoiding obstacles.

Phase Two: Finite State Machine (FSM)
Objective: Implement a Finite State Machine for AI behavior.

Create the basic structure of an FSM.
Define states:
Idle: The agent starts in an idle state.
Scan: The agent scans and detects the goal.
Move: The agent moves towards the goal.
Exit: The agent returns and exits.
Success Criteria: The agent transitions correctly between states based on conditions.

Phase Three: Goal-Oriented Action Planning (GOAP)
Objective: Implement GOAP for dynamic AI behavior planning.

Create the basic structure of GOAP.
Develop an action plan.
Define states:
Idle: The agent starts in an idle state.
Scan: The agent scans and detects the goal.
Move: The agent moves towards the goal.
Exit: The agent returns and exits.
Extra: Handle complex scenarios with double obstacles and enemies.
Success Criteria: The agent can dynamically plan and execute actions to achieve goals.

Phase Four: Hierarchical Task Network (HTN) Planning
Objective: Implement HTN for hierarchical task planning.

Create the basic structure of HTN.
Develop a task plan.
Define states:
Idle: The agent starts in an idle state.
Scan: The agent scans and detects the goal.
Move: The agent moves towards the goal.
Exit: The agent returns and exits.
Extra: Implement object pickup and following behaviors.
Success Criteria: The agent can execute tasks in a hierarchical manner.

Phase Five: Behavior Tree
Objective: Implement a Behavior Tree for complex AI behavior.

Create the basic structure of a behavior tree.
Develop a behavior plan.
Define states:
Idle: The agent starts in an idle state.
Scan: The agent scans and detects the goal.
Move: The agent moves towards the goal.
Exit: The agent returns and exits.
Extra: Update objectives based on events and search for enemies to eliminate.

==========

Phase One: Basic Navigation

Class: RebelBunnieAgent

Purpose: Represents the AI-controlled character (RebelBunnie) in the game world. It's the core class that handles the character's movement and interaction with the navigation system.
Functionality:
MoveTo(Vector3 targetPosition): Instructs the agent to move to a specific location in the game world.
IsAtDestination(): Checks if the agent has reached its current destination.
SetSpeed(float speed): Sets the agent's movement speed.
StopMovement(): Stops the agent's movement.
NavMeshAgent (Unity Component): Utilizes Unity's built-in NavMeshAgent component for pathfinding and obstacle avoidance. This component handles the low-level navigation details.
Class: NavigationManager (Conceptual - Could be part of a service or a utility class)

Purpose: Handles global navigation-related tasks that might be needed by multiple agents or systems. It acts as a central point for navigation queries.
Functionality:
FindPath(Vector3 start, Vector3 end): Calculates a path between two points in the game world. (In practice, this might be handled directly by the NavMeshAgent or a helper function).
IsPathClear(Vector3 start, Vector3 end): Checks if a direct path between two points is clear of obstacles. (Similar to FindPath, this might be handled by the NavMeshAgent or a helper function).
Phase Two: Finite State Machine (FSM)

Class: FSM

Purpose: The core class that manages the Finite State Machine. It's responsible for holding the states, tracking the current state, and handling state transitions.
Functionality:
AddState(State state): Adds a state to the FSM.
SetInitialState(State state): Sets the starting state of the FSM.
ChangeState(State state): Transitions the FSM to a new state.

Update(): Called every frame to check for state transitions and update the current state.
Class: State

Purpose: An abstract base class for all states in the FSM. It defines the common interface that all states must implement.
Functionality:
Enter(): Called when the state is entered.
Execute(): Called every frame while the FSM is in this state.
Exit(): Called when the state is exited.
CheckTransitions(): Checks if the conditions for a state change are met.
Class: IdleState

Purpose: A concrete state representing the "Idle" behavior of the agent.
Functionality:
Enter(): Actions to perform when entering the idle state.
Execute(): Actions to perform while in the idle state (e.g., do nothing, play an idle animation).
Exit(): Actions to perform when exiting the idle state.
CheckTransitions(): Checks if the agent should transition to another state (e.g., if the goal is detected).
Class: ScanState

Purpose: A concrete state representing the "Scanning" behavior of the agent.
Functionality:
Enter(): Actions to perform when entering the scan state.
Execute(): Actions to perform while scanning (e.g., look around for the goal).
Exit(): Actions to perform when exiting the scan state.
CheckTransitions(): Checks if the agent should transition to another state (e.g., if the goal is found).
Class: MoveState

Purpose: A concrete state representing the "Moving" behavior of the agent.
Functionality:
Enter(): Actions to perform when entering the move state.
Execute(): Actions to perform while moving (e.g., move towards the goal).
Exit(): Actions to perform when exiting the move state.
CheckTransitions(): Checks if the agent should transition to another state (e.g., if the goal is reached).
Class: ExitState

Purpose: A concrete state representing the "Exiting" behavior of the agent.
Functionality:
Enter(): Actions to perform when entering the exit state.
Execute(): Actions to perform while exiting (e.g., move towards the exit point).
Exit(): Actions to perform when exiting the exit state.
CheckTransitions(): Checks if the agent should transition to another state (e.g., if the exit is reached).

==========

Here's the documentation for the AISwitchGroupsEditor class:

Class Documentation: AISwitchGroupsEditor

File: AISwitchGroupsEditorWindows.cs

Namespace: RebelRabbit.AI.Editor

Purpose:

The AISwitchGroupsEditor class is a custom Unity Editor window designed to simplify the process of managing and activating different AI groups within a scene. It allows developers to quickly switch between different AI setups by activating only the selected AI group and deactivating all others. This is particularly useful for testing and debugging different AI behaviors or scenarios without manually enabling/disabling GameObjects in the scene.

Key Features:

AI Group Management: Automatically finds all GameObjects in the scene tagged with "AI".
AI Selection: Provides a dropdown menu to select a specific AI group from the list.
Activation/Deactivation: Allows the user to activate the selected AI group and deactivate all other AI groups with a single button click.
Editor Window: Implements a custom Editor window that can be accessed from the Unity menu (Tools/AI/Switch AI Groups).
Sort: Sort the list of AI by name.
Class Structure:

targetAIName (string):
Purpose: Stores the name of the tag used to identify AI GameObjects.
Default Value: "AI"
Usage: Used by the FindAIObjects() method to search for AI GameObjects.
selectedAI (GameObject):
Purpose: Stores the currently selected AI GameObject.
Usage: Used to track which AI group is selected in the dropdown menu.
aiObjects (List):
Purpose: Stores a list of all AI GameObjects found in the scene.
Usage: Populated by the FindAIObjects() method.
selectedAIIndex (int):
Purpose: Stores the index of the currently selected AI in the aiObjects list.
Usage: Used to track the selected AI in the dropdown menu.
Methods:

ShowWindow() (static void):

Purpose: Creates and displays the AISwitchGroupsEditor window.
Usage: Called when the user selects Tools/AI/Switch AI Groups from the Unity menu.

[MenuItem("Tools/AI/Switch AI Groups")]: This attribute adds the menu item to the Unity Editor.
GetWindow<AISwitchGroupsEditor>("AI Switcher"): This method creates or retrieves the AISwitchGroupsEditor window instance.
OnEnable() (private void):

Purpose: Called when the Editor window is enabled or loaded.
Usage: Calls FindAIObjects() to populate the list of AI GameObjects when the window is opened.
OnGUI() (private void):

Purpose: Handles the drawing of the Editor window's user interface.
Usage:
Displays a label: "AI Group Switcher".
If AI objects are found:
Creates a dropdown menu (EditorGUILayout.Popup) to select an AI group.
Updates selectedAI based on the selected index.
If no AI objects are found:
Displays a message: "No AI objects found with the 'AI' tag."
Creates a button: "Activate Selected AI".
Displays the selected AI object.
Calls ActivateSelectedAI() when the button is clicked.
FindAIObjects() (private void):

Purpose: Finds all GameObjects in the scene with the "AI" tag and populates the aiObjects list.
Usage:
Clears the aiObjects list.
Uses GameObject.FindGameObjectsWithTag("AI") to find all AI GameObjects.
Adds the found GameObjects to the aiObjects list.
Sort the list by name.
Sets the selectedAIIndex to 0 and selectedAI to the first element if AI objects are found.
Sets the selectedAIIndex to -1 and selectedAI to null if no AI objects are found.
ActivateSelectedAI() (private void):

Purpose: Deactivates all AI GameObjects and activates the selected AI GameObject.
Usage:
Checks if an AI is selected (selectedAI != null).
If no AI is selected, logs a warning: "No AI selected."
Deactivates all AI GameObjects in the aiObjects list using aiObject.SetActive(false).
Activates the selected AI GameObject using selectedAI.SetActive(true).
How It Works:

Finding AI Groups: When the window is opened, FindAIObjects() searches the scene for GameObjects with the "AI" tag and adds them to the aiObjects list.
Selecting an AI Group: The OnGUI() method creates a dropdown menu that lists the names of the AI GameObjects. The user can select an AI group from this menu.

Activating the Selected AI: When the user clicks the "Activate Selected AI" button, ActivateSelectedAI() deactivates all AI GameObjects and then activates the one that was selected.
Sort: The list of AI is sorted by name.
Usage in the RebelBunnie-AI Project:

This tool is very useful in the RebelBunnie-AI project because it allows developers to easily switch between different AI configurations or test different AI behaviors. For example, you might have different AI groups for:

Testing basic navigation.
Testing FSM behavior.
Testing GOAP planning.
Testing HTN planning.
Testing Behavior Tree behavior.
By using this tool, you can quickly activate the AI group you want to test and deactivate the others, making the testing process much more efficient.

Junior Developer Notes:

Editor Scripting: This script demonstrates how to create custom Editor windows in Unity.
EditorGUILayout: This class is used to create UI elements in Editor windows.
GameObject.FindGameObjectsWithTag(): This method is used to find GameObjects with a specific tag.
SetActive(): This method is used to activate or deactivate GameObjects.
[MenuItem]: This attribute is used to add menu items to the Unity Editor.
OrderBy: This method is used to sort a list.
AddComponentMenu: This attribute is used to add a component to the Add Component menu.
This documentation provides a comprehensive overview of the AISwitchGroupsEditor class. It should be very helpful for anyone working on the RebelBunnie-AI project, especially junior developers who are learning about Unity Editor scripting. Let me know if you have any other questions.

==========
Here's the documentation for the AISwitchGroupsEditor class:

Class Documentation: AISwitchGroupsEditor

File: AISwitchGroupsEditorWindows.cs

Namespace: RebelRabbit.AI.Editor

Purpose:

The AISwitchGroupsEditor class is a custom Unity Editor window designed to simplify the process of managing and activating different AI groups within a scene. It allows developers to quickly switch between different AI setups by activating only the selected AI group and deactivating all others. This is particularly useful for testing and debugging different AI behaviors or scenarios without manually enabling/disabling GameObjects in the scene.

Key Features:

AI Group Management: Automatically finds all GameObjects in the scene tagged with "AI".
AI Selection: Provides a dropdown menu to select a specific AI group from the list.
Activation/Deactivation: Allows the user to activate the selected AI group and deactivate all other AI groups with a single button click.
Editor Window: Implements a custom Editor window that can be accessed from the Unity menu (Tools/AI/Switch AI Groups).
Sort: Sort the list of AI by name.
Class Structure:

targetAIName (string):
Purpose: Stores the name of the tag used to identify AI GameObjects.
Default Value: "AI"
Usage: Used by the FindAIObjects() method to search for AI GameObjects.
selectedAI (GameObject):
Purpose: Stores the currently selected AI GameObject.
Usage: Used to track which AI group is selected in the dropdown menu.
aiObjects (List):
Purpose: Stores a list of all AI GameObjects found in the scene.
Usage: Populated by the FindAIObjects() method.
selectedAIIndex (int):
Purpose: Stores the index of the currently selected AI in the aiObjects list.
Usage: Used to track the selected AI in the dropdown menu.
Methods:

ShowWindow() (static void):

Purpose: Creates and displays the AISwitchGroupsEditor window.
Usage: Called when the user selects Tools/AI/Switch AI Groups from the Unity menu.
[MenuItem("Tools/AI/Switch AI Groups")]: This attribute adds the menu item to the Unity Editor.
GetWindow<AISwitchGroupsEditor>("AI Switcher"): This method creates or retrieves the AISwitchGroupsEditor window instance.
OnEnable() (private void):

Purpose: Called when the Editor window is enabled or loaded.
Usage: Calls FindAIObjects() to populate the list of AI GameObjects when the window is opened.
OnGUI() (private void):

Purpose: Handles the drawing of the Editor window's user interface.

Usage:
Displays a label: "AI Group Switcher".
If AI objects are found:
Creates a dropdown menu (EditorGUILayout.Popup) to select an AI group.
Updates selectedAI based on the selected index.
If no AI objects are found:
Displays a message: "No AI objects found with the 'AI' tag."
Creates a button: "Activate Selected AI".
Displays the selected AI object.
Calls ActivateSelectedAI() when the button is clicked.
FindAIObjects() (private void):

Purpose: Finds all GameObjects in the scene with the "AI" tag and populates the aiObjects list.
Usage:
Clears the aiObjects list.
Uses GameObject.FindGameObjectsWithTag("AI") to find all AI GameObjects.
Adds the found GameObjects to the aiObjects list.
Sort the list by name.
Sets the selectedAIIndex to 0 and selectedAI to the first element if AI objects are found.
Sets the selectedAIIndex to -1 and selectedAI to null if no AI objects are found.
ActivateSelectedAI() (private void):

Purpose: Deactivates all AI GameObjects and activates the selected AI GameObject.
Usage:
Checks if an AI is selected (selectedAI != null).
If no AI is selected, logs a warning: "No AI selected."
Deactivates all AI GameObjects in the aiObjects list using aiObject.SetActive(false).
Activates the selected AI GameObject using selectedAI.SetActive(true).
How It Works:

Finding AI Groups: When the window is opened, FindAIObjects() searches the scene for GameObjects with the "AI" tag and adds them to the aiObjects list.
Selecting an AI Group: The OnGUI() method creates a dropdown menu that lists the names of the AI GameObjects. The user can select an AI group from this menu.
Activating the Selected AI: When the user clicks the "Activate Selected AI" button, ActivateSelectedAI() deactivates all AI GameObjects and then activates the one that was selected.
Sort: The list of AI is sorted by name.
Usage in the RebelBunnie-AI Project:

This tool is very useful in the RebelBunnie-AI project because it allows developers to easily switch between different AI configurations or test different AI behaviors. For example, you might have different AI groups for:

Testing basic navigation.
Testing FSM behavior.
Testing GOAP planning.

Testing HTN planning.
Testing Behavior Tree behavior.
By using this tool, you can quickly activate the AI group you want to test and deactivate the others, making the testing process much more efficient.

Junior Developer Notes:

Editor Scripting: This script demonstrates how to create custom Editor windows in Unity.
EditorGUILayout: This class is used to create UI elements in Editor windows.
GameObject.FindGameObjectsWithTag(): This method is used to find GameObjects with a specific tag.
SetActive(): This method is used to activate or deactivate GameObjects.
[MenuItem]: This attribute is used to add menu items to the Unity Editor.
OrderBy: This method is used to sort a list.
AddComponentMenu: This attribute is used to add a component to the Add Component menu.
This documentation provides a comprehensive overview of the AISwitchGroupsEditor class. It should be very helpful for anyone working on the RebelBunnie-AI project, especially junior developers who are learning about Unity Editor scripting. Let me know if you have any other questions.